

White Paper

Introduction to neural networks for Image Analytics

Introduction

Neural Networks are a class of models within the general machine learning literature. They are a specific set of algorithms that has revolutionized the field of image analytics. They are inspired by biological neural networks and the current so-called deep neural networks have proven to work quite very well for multiple use cases based on image processing and recognition.

Neural Networks are themselves general function approximations, that is why they can be applied to literally almost any image and computer vision problem where the problem is about learning a complex mapping from the input to the output space.

Some of the top tasks that can be accomplished with the help of neural networks are

1. Recognizing patterns: Objects in real scenes, Facial identities or facial expressions, spoken words
2. Recognizing anomalies: unusual sequence of activities or unusual pattern identification
3. Prediction: Future predictions based on inputs and machine level understanding

Over the years, multiple neural network architecture has been developed and implemented across various use cases. We would like to focus on some of the most popular and prominent architectures of neural networks with wide applications across image analytics.

Convolved Neural Networks (CNNs)

Convolved neural networks was developed by Yann LeCun and his associates in 1998 (originally developed a good recognizer for handwriting digit called as LeNet). It used back propagation in a feedforward net with multiple hidden layers, many maps of replicated units in each layer and pooling the outputs of nearby replicated units. Thus, it has a wide net that can cope with several characters at once even if they overlap.

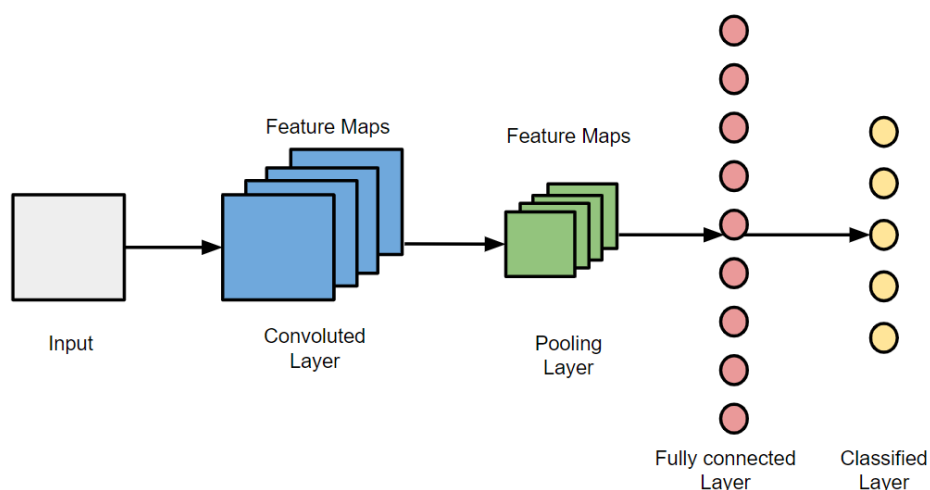


Fig. 1 Convoluted Neural Network architecture

CNNs are quite different from most other neural networks architectures available in the market. They are primarily used in image processing and classification but can also be used across other input types such as sound and audio.

The key is to understand the working of Convoluted Neural Networks (CNNs). The process begins with the input data fed through convoluted layers (as shown in Fig. 1) instead of traditional normal layers, where not all nodes are connected to all nodes. These convoluted layers have a tendency to shrink when they become deeper. CNNs also involve pooling layers. Pooling is a way to filter out details and one of the commonly found pooling techniques is max pooling. It is a pooling operation that calculates value for the patches of a feature map, and uses it to create a downsampled (pooled) feature map. It is usually used after the convolutional layer.

Recurrent Neural Networks (RNNs)

Originally introduced by Jeffrey Elman in the year 1990, Recurrent Neural Networks (RNNs) are basically perceptrons. However, they are slightly different from traditional perceptrons. Unlike traditional perceptrons which are stateless, RNNs have connections between passes, connections through time.

They are very powerful because they combine two key properties namely distributed hidden state that allows them to store a lot of information about the past efficiently and second one is non-linear dynamics that allow them to update their hidden state in a complicated manner.

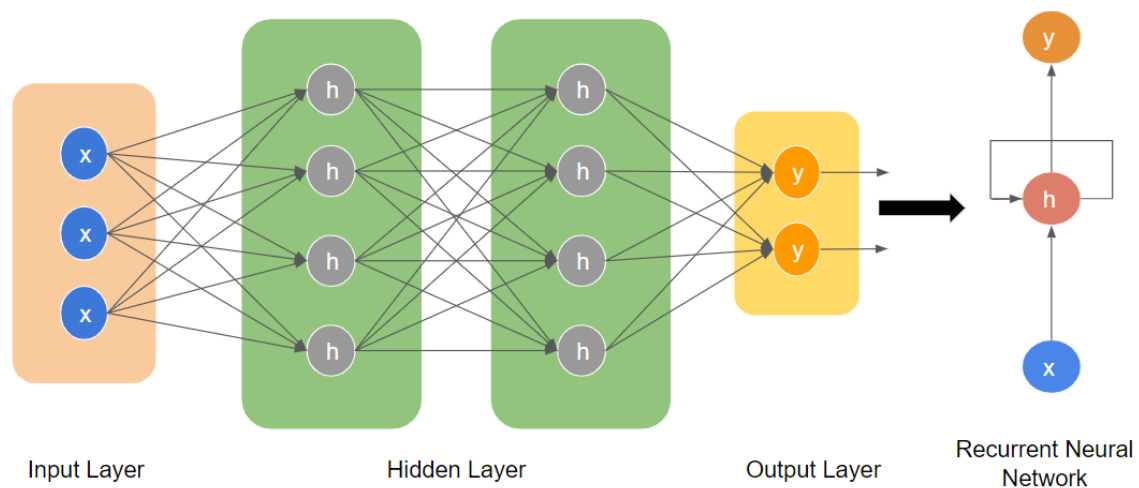


Fig.2 Recurrent Neural Network architecture

In Spite of being such a powerful neural network architecture, it has one major issue of vanishing (or exploding) gradient. It's the problem where depending on the activation function used, information rapidly gets lost over time. This will become a major concern where information from the past is stored; if the weight reaches a value of 0 or 1000000, the previous state won't be informative.

RNNs can in principle be used in many fields as most forms of data don't actually have a timeline (except sound or video). In general, recurrent networks are a great choice for advancing or completing information.

Long / Short Term Memory (LSTMs)

Hochreiter & Schmidhuber solved the problem of getting a RNN to remember things for a long time by building what are known as long-short term memory networks (LSTMs). LSTM networks try to combat the vanishing gradient problem by introducing gates and explicitly defining memory cells. This memory cell stores the previous values and holds onto it unless a “forgot gate” tells this cell to forget those values. (Fig.3)

In short, LSTM has an input as well as an output gate. Through the input gate we can add new stuff or items in the memory cell and through the output gate we can determine whether to pass along the vector from this cell to the next hidden state.

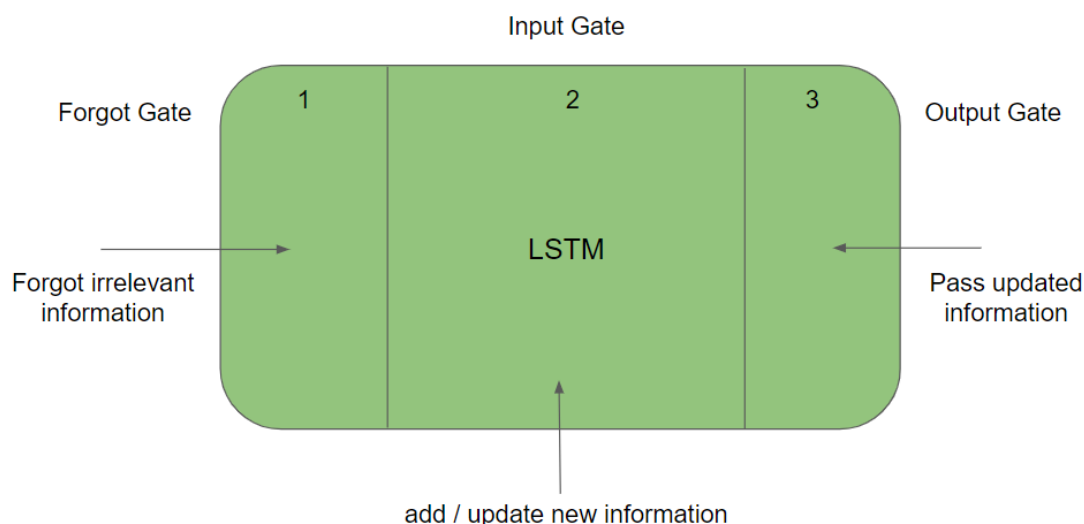


Fig.3 Long-Short-Term-Memory Network

One of the biggest advantages of LSTM is the addition of a cell layer that ensures the transfer of hidden state information from one iteration to the next with ease. In comparison to other neural network architectures, LSTM leads to many more successful runs with higher learnability. It also solves complex, artificial long time lag tasks that have never been solved by previous recurrent network algorithms.

Gated Recurrent Units (GRUs)

Gated Recurrent Units (GRUs) are a slight variation on LSTMs. What makes them different from LSTMs is that GRUs don’t need the cell layer to pass the value along.

Unlike LSTM, GRUs consist of only three gates and do not maintain an internal cell state. The information which is stored in the Internal Cell State in an LSTM recurrent unit is incorporated into the hidden state of the Gated Recurrent Unit. This collective information is passed onto the next Gated Recurrent Unit. The different gates of a GRU are as described below: -

1. Update Gate: It determines how much of the past information needs to be passed along into the future. It is analogous to output gate in LSTMs
2. Reset Gate: It determines how much of the past information needs to be forgotten. It is analogous to the input gate in LSTMs
3. Current Memory Gate: It is incorporated into the Reset Gate and is used to introduce some nonlinearity into the input and to also make the input Zero-mean. Another reason is to reduce the effect that previous information has on the current information that is being passed into the future.

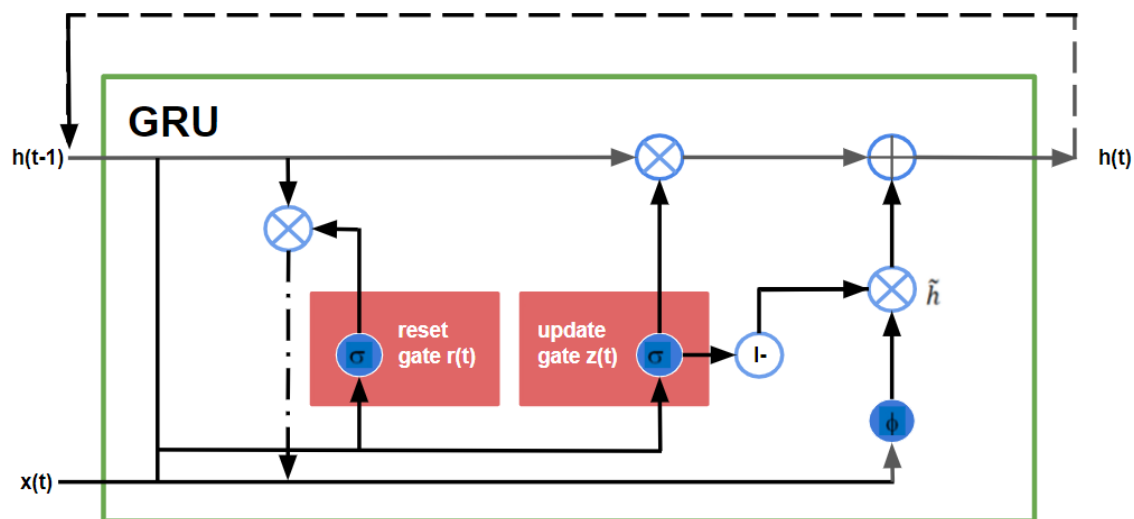


Fig.4 Gated recurrent Units (GRUs)

In most cases, GRUs function similarly to LSTMs with the biggest advantage being that GRUs are slightly faster and relatively easier to run.

Generative Adversarial Network (GANs)

Ian Goodfellow introduced a new breed of neural network, in which 2 networks work together. Generative Adversarial Networks (GANs) consist of any two networks (although often a combination of Feed Forwards and Convolutional Neural Nets), with one tasked to generate content (generative) and the other has to judge content (discriminative). The discriminative model has the task of determining whether a given image looks natural (an image from the dataset) or looks like it has been artificially created. The task of the generator is to create natural looking images that are similar to the original data distribution.

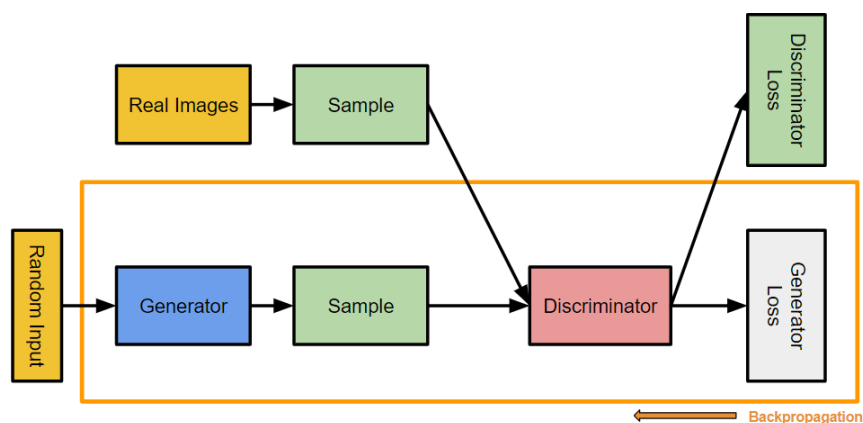


Fig.5 Generative Adversarial Networks (GANs)

GANs are now a very active topic of research and there have been many different types of GAN implementation. Some of the important ones that are actively being used currently are described below:

- Vanilla GAN: This is the simplest type of GAN. Here, the Generator and the Discriminator are simple multi-layer perceptrons. In vanilla GAN, the algorithm is really simple, it tries to optimize the mathematical equation using stochastic gradient descent.
- Conditional GAN (CGAN): CGAN can be described as a deep learning method in which some conditional parameters are put into place. In CGAN, an additional parameter 'y' is added to the Generator for generating the corresponding data. Labels are also put into the input to the Discriminator in order for the Discriminator to help distinguish the real data from the fake generated data.
- Deep Convolutional GAN (DCGAN): DCGAN is one of the most popular and also the most successful implementation of GAN. It is composed of ConvNets in place of multi-layer perceptrons. The ConvNets are implemented without max pooling, which is in fact replaced by convolutional stride. Also, the layers are not fully connected.
- Laplacian Pyramid GAN (LAPGAN): The Laplacian pyramid is a linear invertible image representation consisting of a set of band-pass images, spaced an octave apart, plus a low-frequency residual. This approach uses multiple numbers of Generator and Discriminator networks and different levels of the Laplacian Pyramid. This approach is mainly used because it produces very high-quality images. The image is down-sampled at first at each layer of the pyramid and then it is again up-scaled at each layer in a backward pass where the image acquires some noise from the Conditional GAN at these layers until it reaches its original size.
- Super Resolution GAN (SRGAN): SRGAN as the name suggests is a way of designing a GAN in which a deep neural network is used along with an adversarial network in order to produce higher resolution images. This type of GAN is particularly useful in optimally up-scaling native low-resolution images to enhance its details, minimizing errors while doing so.

GANs are already being applied in industry for a variety of applications ranging from interactive image editing, 3D shape estimation, drug discovery, semi-supervised learning to robotics.

Conclusion

To conclude, neural networks are one of the most beautiful programming paradigms ever invented. With the help of neural networks, we don't tell the computers on how to solve our problem. Instead, it learns from the observational data, figuring out its own solution to the problem in hand. With the help of this network architecture many important problems in image analytics and computer vision can be solved with ease. We will discuss some of its applications in our next blogs.

References

<https://openai.com/blog/generative-models/>
<https://data-notes.co/a-gentle-introduction-to-neural-networks-for-machine-learning-d5f3f8987786>
https://en.wikipedia.org/wiki/Generative_adversarial_network
<http://jalamar.github.io/visual-interactive-guide-basics-neural-networks/>
<http://colah.github.io/posts/2015-08-Understanding-LSTMs/>